Core Concepts

> Part of the Streamlit curriculum. This document covers Streamlit's core execution model, caching, widgets, layout, session state, and forms.

1. What Streamlit is, and its core mental model

Streamlit turns a plain Python script into a web app: you write top-to-bottom procedural code that loads data and calls Streamlit functions to render UI elements, and Streamlit handles turning that into a browser-based interface — no HTML, CSS, or JavaScript required, no separate frontend build step, and no need to design an HTTP contract, since the app has direct in-process access to whatever data/objects the script computes.

The core thing to understand before anything else: Streamlit **reruns your entire script, top to bottom, every time the user interacts with a widget** (changes a dropdown, clicks a button, types in a text field). There is no persistent state you manage yourself by default — every plain variable is recomputed on every rerun. This has direct consequences that shape almost everything else in this document:

- Anything expensive (loading a large file, an expensive computation) needs to be **cached** (§2), or it will re-run on every single interaction, making the app feel slow.
- The current value of a widget (e.g., which option is selected in a dropdown) is simply the widget function's *return value* on that rerun — you read it by assigning the return value to a variable, not by managing state manually.
- Anything you *do* want to persist across reruns that isn't naturally derived from a widget's current value needs **session state** (§5) — an explicit escape hatch from the rerun-from-scratch model.

2. Caching

```python
import streamlit as st
import pandas as pd

@st.cache_data
def load_dataset():
    return pd.read_csv("path/to/computed_output.csv")
```

`@st.cache_data` memoizes a function's return value based on its arguments — the underlying file is read once, and subsequent calls (across reruns) return the cached result instantly, until the function's arguments change or the cache is explicitly invalidated. This is the direct Streamlit equivalent of a plain Python `functools.lru_cache` used in a backend service (see Python API Backend, §1) — same idea, framework-specific mechanism.

2.1 `st.cache_data` vs. `st.cache_resource`

Streamlit has two distinct caching decorators, and using the wrong one causes subtle bugs:

- `st.cache_data` — for data that should be **copied** on every cache hit (DataFrames, lists, dicts, anything you'd be comfortable serializing). Streamlit returns a fresh copy each time, so one part of the app mutating the returned object can't accidentally corrupt what another part of the app sees.
- `st.cache_resource` — for objects that should be **shared** as the exact same instance across reruns and users (a database connection, a loaded ML model object, anything expensive to create and safe/intended to be reused directly rather than copied).

```
@st.cache_resource
def get_model():
    return load_expensive_model_from_disk()
```

Using `cache_data` for something like a database connection would try to copy it (usually failing or behaving unexpectedly); using `cache_resource` for a DataFrame you intend to filter locally risks every caller sharing and potentially mutating the exact same underlying object.

2.2 Cache expiry

Both decorators accept a `ttl` (time-to-live) argument for data that should periodically refresh even without an explicit code change:

```
@st.cache_data(ttl=3600)   # re-fetch at most once per hour
def load_dataset():
    ...
```

3. Layout and widgets

```python
st.set_page_config(page_title="My Dashboard", layout="wide")

st.sidebar.title("Navigation")
page = st.sidebar.radio("Go to", ["Overview", "Detail"])

col1, col2 = st.columns(2)
with col1:
    st.metric(label="Total", value="1,234")
with col2:
    st.metric(label="% Change", value="+12.3%")

selected = st.selectbox("Choose one", options=["A", "B", "C"])
selected_many = st.multiselect("Choose several", options=["A", "B", "C"], default=["A"])
value = st.slider("Pick a number", min_value=0, max_value=100, value=50)
text = st.text_input("Enter a value")
uploaded = st.file_uploader("Upload a CSV", type="csv")

tab1, tab2 = st.tabs(["Chart", "Table"])
with tab1:
    st.line_chart(df)
with tab2:
    st.dataframe(df, use_container_width=True)

with st.expander("More detail"):
    st.dataframe(df, use_container_width=True)

with st.container():
    st.write("Grouped content, useful for applying one style/border to several elements together")
```

The widgets that matter most for a data-exploration dashboard: `st.selectbox` (pick one), `st.multiselect` (pick several), `st.radio` (pick one, shown as visible options rather than a dropdown — good for a small, fixed set like a page/view switcher), `st.slider` (a numeric range), `st.text_input` / `st.file_uploader` (free-form input), `st.columns` (side-by-side layout), `st.tabs` (a tabbed layout within one page), `st.expander` (collapsible section, useful for optional/secondary detail), `st.container` (a grouping element with no visual effect on its own, useful for applying layout/styling to a group), and `st.metric` / `st.dataframe` for displaying numbers and tables.

Every widget function's *return value* is the current selection — read it directly into a variable and branch on it. Every widget also accepts a `key=` argument, which matters once you have more than one widget of the same type on a page, or need to reference a widget's current value from `st.session_state` (§5) — Streamlit needs a unique key to keep multiple widgets' state distinct.

A common top-level structure for a multi-page dashboard, given the "one script reruns top-to-bottom" model (§1), is a single sidebar control selecting which "page" to show, followed by an `if page == "...":` / `elif` chain —

there's no built-in routing the way a web framework has, just conditional rendering within one script. See §6 for Streamlit's alternative, native multi-page-app support.

4. Forms

By default, every widget triggers an immediate rerun the moment its value changes — fine for a single control, but wasteful if a user needs to set five different filters before the "real" work (a heavy query, a chart re-render) should happen. `st.form` batches multiple widgets so the script only reruns once, when the user explicitly submits:

```
with st.form("filters"):
    country = st.selectbox("Country", options=COUNTRIES)
    year_range = st.slider("Year range", 1990, 2024, (2000, 2020))
    submitted = st.form_submit_button("Apply")

if submitted:
    st.write(f"Filtering for {country}, {year_range}")
```

Nothing inside the `with st.form(...)` block triggers a rerun on its own — only clicking the `st.form_submit_button` does, at which point `submitted` becomes `True` for that one rerun and all the widgets' current values are available together.

5. Session state

`st.session_state` is Streamlit's explicit mechanism for state that should persist across reruns, independent of any single widget's current value — a dictionary-like object that survives from one rerun to the next for the same user session:

```
if "counter" not in st.session_state:
    st.session_state.counter = 0

if st.button("Increment"):
    st.session_state.counter += 1

st.write(f"Count: {st.session_state.counter}")
```

Without `session_state`, a plain variable `counter = 0` would reset to `0` on every single rerun — there'd be no way to "remember" it was previously incremented. Reach for `session_state` whenever you need something to persist that *isn't* simply "whatever a widget's current value happens to be" — a running total, a flag indicating a multi-step workflow's current stage, or a value one part of the page needs to set and another part needs to read later in the same rerun (or a subsequent one).

6. Multi-page apps

Beyond the sidebar-radio-and-`if`/`elif` pattern (§3), Streamlit has native multi-page-app support: any script placed in a `pages/` directory next to your main script automatically becomes a separate page, with Streamlit generating navigation between them for you:

```
my_app/
├── app.py            # the main/home page
└── pages/
    ├── 1_Overview.py
    └── 2_Detail.py
```

The leading number in each filename (`1_`, `2_`) controls display order in the auto-generated navigation. This is a good option once a dashboard's pages are substantial enough to warrant their own files rather than branches of one `if`/`elif` chain — a maintainability trade-off, not a functional requirement, since both approaches produce a working multi-page experience.

7. Handling missing or not-yet-available data gracefully

A dashboard reading from files a separate pipeline produces needs to handle "that file doesn't exist yet" as a normal, expected condition — not a crash:

```python
@st.cache_data
def load_dataset():
    if not os.path.exists(PATH):
        return None
    return pd.read_csv(PATH)

df = load_dataset()
if df is None:
    st.warning("This data hasn't been generated yet. Run the upstream pipeline step first.")
else:
    # render the page normally
    ...
```

This is the same "degrade gracefully rather than crash" principle discussed for a backend API (Python API Backend, §2.2) — here expressed as an in-page message instead of an HTTP status code, since there's no separate client to interpret a status code; the message *is* the entire response.

8. Core libraries summary

Library	Used for
Streamlit	the app framework — page config, widgets, caching, layout, session state
pandas	loading and shaping data for display

See also

- Streamlit index
- Charting & Visualization — rendering the data these concepts load and filter